

Topic 3: Decision Trees and Random Forests

0.1 Decision Trees

0.1.1 Playing Twenty Questions with Data

Intuition & Analogy

Think of the game *Twenty Questions*: one player thinks of an object, and the other narrows it down with yes/no questions — “Is it alive?”, “Is it bigger than a car?”. A good player asks the questions that eliminate the most uncertainty first. A **decision tree** plays exactly this game with data: it asks a sequence of simple yes/no questions about the features (“Is hours studied > 5.5 ?”) until it is confident enough to announce a class.

Definition: Decision Tree

A decision tree is a flowchart-like model. Each **internal node** tests one feature against a threshold (e.g. “score ≤ 61 ?”), each **branch** corresponds to an answer (yes/no), and each **leaf node** outputs a predicted class. A new example is classified by starting at the **root** and following the branches that match its feature values until reaching a leaf.

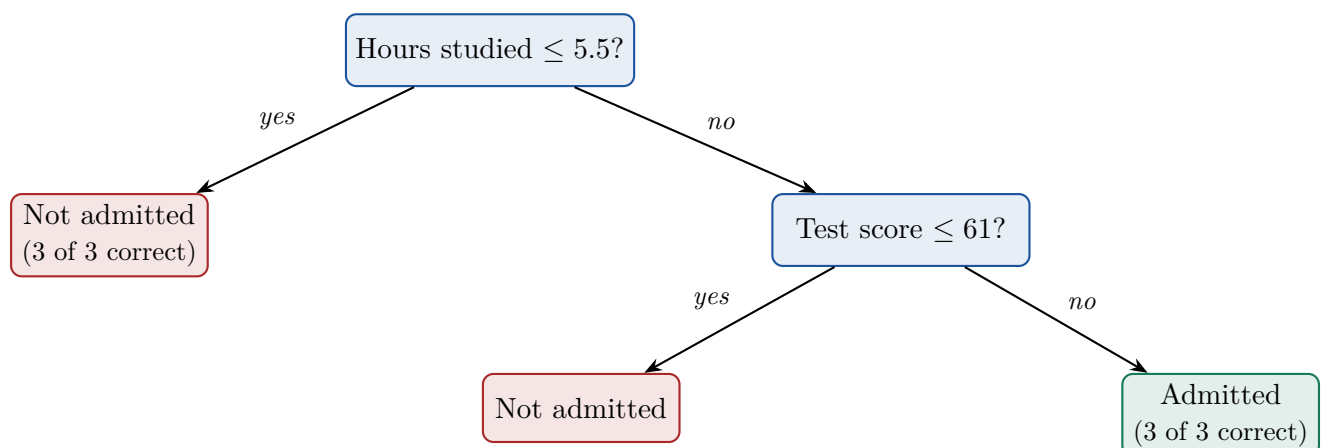


Figure 1: A small decision tree for the bootcamp data (see Topic 1). Classification means following the arrows: an applicant with 8 hours and score 70 goes right at the root ($8 > 5.5$), right again ($70 > 61$), and lands in the *Admitted* leaf.

0.1.2 How Does the Tree Choose Its Questions?

The heart of the algorithm is deciding, at every node, *which feature to split on and at what threshold*. The guiding idea: a good question separates the classes as cleanly as possible. We need a number that measures “messiness” of a group of labels, so we can pick the question that reduces messiness the most. Two standard measures exist.

Measure 1: Gini Impurity

Definition: Gini Impurity

For a node containing a mix of classes, where p_c is the fraction of examples belonging to class c :

$$\text{Gini} = 1 - \sum_c p_c^2 \quad (1)$$

Gini answers the question: *if I pick one example at random from this node and guess its label by drawing another random label from the same node, how often am I wrong?*

Feel it with numbers (binary case):

- A **pure** node, all class 1: $\text{Gini} = 1 - (1^2 + 0^2) = 0$. No messiness at all.
- A perfect **50/50** mix: $\text{Gini} = 1 - (0.5^2 + 0.5^2) = 0.5$. Maximum messiness for two classes.
- A 90/10 mix: $\text{Gini} = 1 - (0.81 + 0.01) = 0.18$. Mostly clean.

Measure 2: Entropy and Information Gain

Definition: Entropy

$$\text{Entropy} = - \sum_c p_c \log_2 p_c \quad (2)$$

Entropy measures uncertainty in *bits*. A pure node has entropy 0 (no surprise); a 50/50 binary node has entropy 1 (a full coin-flip of surprise).

The **information gain** of a split is the entropy before the split minus the (size-weighted) average entropy of the child nodes:

$$\text{Gain} = \text{Entropy}_{\text{parent}} - \sum_{\text{children } j} \frac{N_j}{N} \text{Entropy}_j \quad (3)$$

The tree tries every feature and every sensible threshold, and greedily picks the split with the highest gain (or, equivalently, the biggest drop in Gini). It then repeats the process inside each child node, growing the tree recursively until nodes are pure or a stopping rule kicks in.

Worked Example: Computing Information Gain by Hand

Take the six bootcamp applicants: three Admitted, three Not admitted.

Step 1 — Entropy of the root. With $p_0 = p_1 = 0.5$:

$$\text{Entropy} = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = -(0.5 \times (-1) + 0.5 \times (-1)) = 1 \text{ bit.}$$

Maximum uncertainty — the node is a coin flip.

Step 2 — Try the split “hours ≤ 5.5 ”.

- Left child (hours ≤ 5.5): applicants A, B, C — labels $\{0, 0, 0\}$. Pure! Entropy = 0.

- Right child (hours > 5.5): applicants D, E, F — labels {1, 1, 1}. Pure! Entropy = 0.

Step 3 — Information gain.

$$\text{Gain} = 1 - \left(\frac{3}{6} \times 0 + \frac{3}{6} \times 0 \right) = 1 \text{ bit.}$$

This single question removes *all* uncertainty — the best possible split. The tree stops immediately with two pure leaves. (Real datasets are rarely this tidy, which is why trees usually need several levels of questions.)

0.1.3 Overfitting: Trees That Memorise

Left unchecked, a tree keeps splitting until every leaf is perfectly pure — even if that means creating a private leaf for every single noisy training point. Such a tree scores 100% on training data and performs poorly on new data: classic **overfitting**. It is like a student who memorises last year's exam answers word for word instead of learning the subject.

Two families of remedies exist:

- **Pre-pruning (early stopping):** limit the tree while growing it — cap the depth (`max_depth`), require a minimum number of samples to split a node (`min_samples_split`) or to form a leaf (`min_samples_leaf`).
- **Post-pruning:** grow the full tree, then cut back branches that do not improve validation performance (`ccp_alpha` in scikit-learn).

0.1.4 Assumptions and Practical Notes

- Trees make almost no statistical assumptions: no linearity, no scaling, no particular distribution.
- Splits are axis-aligned (each question involves a single feature), so diagonal class boundaries are approximated by staircases.
- The greedy split-by-split construction finds a good tree, not necessarily the best possible tree.

0.1.5 Advantages and Disadvantages

Table 1: Strengths and weaknesses of decision trees.

Advantages	Disadvantages
Fully interpretable — rules can be read and explained	Overfits easily if grown without limits
No feature scaling needed	Unstable: a small data change can produce a very different tree
Handles numeric and categorical features	Greedy splitting may miss better global structures
Captures non-linear patterns and feature interactions	Axis-aligned splits struggle with diagonal boundaries
Fast predictions	Biased towards features with many possible split points

Common Mistakes

- **Growing an unlimited tree** and celebrating 100% training accuracy — that number is a symptom of overfitting, not success.
- **Scaling features unnecessarily.** Harmless, but it shows a misunderstanding: trees compare one feature against a threshold, so scale does not matter.
- **Reading feature importances as causation.** Importance means “useful for splitting”, not “causes the outcome”.
- **Trusting a single tree’s stability.** Retrain on slightly different data and the rules may change completely — the very problem random forests were invented to fix.

Key Points

- A decision tree classifies by asking a sequence of yes/no questions about features.
- Splits are chosen to maximise purity, measured by Gini impurity or entropy/information gain.
- Unlimited trees memorise noise; control them with depth limits and pruning.
- Trees are interpretable but unstable — the perfect motivation for the next section.

0.2 Random Forest

0.2.1 The Wisdom of Crowds

Intuition & Analogy

At a village fair, hundreds of people guess the weight of an ox. Individually the guesses are all over the place — some far too high, some far too low — but their *average* lands astonishingly close to the truth. Individual errors point in different directions and cancel out. A **random forest** applies this “wisdom of crowds” to decision trees: instead of trusting one unstable tree, it trains hundreds of *deliberately different* trees and lets them vote.

Definition: Random Forest

A random forest is an **ensemble** of many decision trees. Each tree is trained on a random variation of the data, and the forest’s prediction is the **majority vote** of all its trees (for classification). Randomness is injected in two places: which *rows* each tree sees, and which *features* each split may consider.

The single tree’s weakness — instability — becomes the forest’s fuel. Because each tree errs in its own way, averaging many diverse trees washes out the individual quirks while keeping the shared signal.

0.2.2 Source of Randomness 1: Bagging (Bootstrap Aggregating)

Each tree receives its own **bootstrap sample**: a dataset of the same size as the original, created by drawing rows *with replacement*. Some rows appear twice or more; on average about 37% of rows are left out of any given tree’s sample. Every tree therefore grows up seeing a slightly different world.

The left-out rows for each tree are called **out-of-bag (OOB)** samples. They act as a free, built-in test set: each tree can be evaluated on the rows it never saw, giving the **OOB score** — an honest performance estimate without needing a separate validation split.

0.2.3 Source of Randomness 2: Random Feature Selection

If we only used bagging, all trees would still likely put the single strongest feature at their root, making them correlated — and a crowd of clones is no wiser than one individual. So the forest adds a second twist: *at every split, each tree may only consider a random subset of the features* (commonly $\sqrt{n}$ of the n features). Sometimes the “star” feature is not on the menu, forcing the tree to discover value in other features. This decorrelates the trees and makes the ensemble genuinely diverse.

0.2.4 Putting It Together

1. For each of T trees (e.g. $T = 100$): draw a bootstrap sample of the rows.
2. Grow a deep decision tree on that sample, but at every split consider only a random subset of features.
3. To classify a new point: run it through all T trees and take the majority vote. The fraction of trees voting for a class serves as a probability estimate.

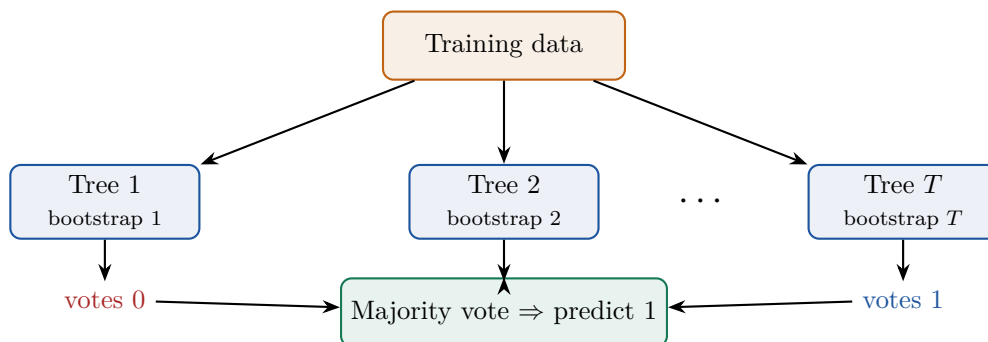


Figure 2: A random forest trains many trees on different bootstrap samples (with random feature subsets at each split) and combines their votes.

Worked Example: Why Voting Beats a Single Tree — a Tiny Calculation

Suppose each individual tree is right 70% of the time, and suppose (idealistically) their errors are independent. Take just $T = 5$ trees and use majority vote. The vote is correct whenever at least 3 trees are correct. Using the binomial distribution:

$$P(\text{majority correct}) = \sum_{j=3}^5 \binom{5}{j} (0.7)^j (0.3)^{5-j} = 0.3087 + 0.3602 + 0.1681 \approx 0.837.$$

Five mediocre voters (70%) combine into a strong committee (83.7%), and with 100 trees the figure climbs above 99% under independence. Real trees are never fully independent — which is exactly why the forest works so hard (bagging + random features) to make them as independent as possible.

Hands-On Python Notebook

The full Python implementation for this topic lives in the companion notebook `03_Decision_Trees_and_Random_Forests.ipynb`. It covers (i) a single decision tree and a random forest on the mini bootcamp dataset, with feature importances, OOB score, and a plot of the learned rules, and (ii) a complete real-data workflow on the kyphosis medical dataset (`kyphosis.csv`): EDA with a pairplot, train/test split, training a `DecisionTreeClassifier`, visualising the tree with `plot_tree`, comparing against a `RandomForestClassifier`, and evaluating both with confusion matrices and classification reports.

0.2.5 Key Hyperparameters

Table 2: The random forest dials worth knowing.

Parameter	Meaning and practical advice
<code>n_estimators</code>	Number of trees. More is better (performance plateaus); costs only computation time. 100–500 is typical.
<code>max_features</code>	Features considered per split. " <code>sqrt</code> " is the standard for classification; lower values increase tree diversity.
<code>max_depth</code>	Depth cap per tree. Forests tolerate deep trees, but capping speeds training and can help on noisy data.
<code>min_samples_leaf</code>	Minimum examples per leaf; raising it smooths predictions.
<code>oob_score</code>	Set <code>True</code> to get the free out-of-bag validation estimate.

0.2.6 Advantages and Disadvantages

Table 3: Strengths and weaknesses of random forests.

Advantages	Disadvantages
Usually strong accuracy with little tuning	Loses the single tree's easy interpretability
Highly resistant to overfitting compared with one tree	Slower to train and predict than a single tree
Handles non-linear patterns and feature interactions	Large memory footprint (hundreds of trees)
No feature scaling required	Can struggle to extrapolate beyond the training data range
Built-in feature importances and OOB validation	Less effective on very sparse, high-dimensional data (e.g. raw text)

Common Mistakes

- **Treating more trees as overfitting risk.** Adding trees does not overfit; it only stabilises the vote. The waste is computational, not statistical.
- **Ignoring the OOB score** and burning data on a separate validation split unnecessarily for small datasets.

- **Reading feature importances uncritically.** They are biased towards high-cardinality features and say nothing about causation.
- **Using a forest when interpretability is legally or ethically required** (e.g. credit decisions) without adding explanation tools.

Key Points

- Random forest = many decision trees + bagging (random rows) + random feature subsets + majority vote.
- Diversity among trees is the whole trick: independent errors cancel out.
- OOB samples provide a free validation estimate.
- It trades the single tree's transparency for substantially better, more stable accuracy.